PROJET PÉDAGOGIQUE · ARCHITECTURE LOGICIELLE

Loxia

Plateforme de location immobilière

Architecture microservices · Java 21 · Spring Boot 3.3 · React 18 · Docker

6 Microservices | Spring Cloud Gateway | PostgreSQL 16 | Docker Compose | React + TypeScript | WebSocket STOMP

2025-2026 · Ugo Durand · Jonas Obrun · Nicolas Ramirez · Cyril Grosjean · Ibrahim Khan · Tarek El Missiry

01 · CONTEXTE

Loxia en 30 secondes

Le projet

Une plateforme de location immobilière qui met en relation **propriétaires** et **locataires**, bâtie sur une vraie architecture distribuée.

Un seul compte

Tout utilisateur peut à la fois **publier des annonces** et **candidater** à des locations. Le basculement se fait via un simple toggle dans l'interface.

Objectif pédagogique

Démontrer une architecture microservices **réaliste** — sans sur-ingénierie, déployable en une commande.

6
Services

5
Bases de données

9
Conteneurs Docker

1 cmd
Pour tout lancer

02 · ARCHITECTURE

Vue d'ensemble — 5 services, 1 gateway

🌐 **Frontend React — Nginx · :3000**

↓

Authorization: Bearer <JWT>

🔒 **API Gateway — Spring Cloud · :8080 · Valide le JWT · Route les requêtes · Bloque /internal/****

↓

X-User-Id · X-User-Email · X-User-FullName

🔑
auth-service
:8081
Inscription · Login
JWT · Profil

🏠
catalog-service
:8082
Annonces
Recherche · Lock

📋
rental-service
:8083
Candidatures
Accept / Reject

🔔
notification-service
:8084
Notifs in-app
Unread count

💬
chat-service
:8085
Messagerie
WebSocket STOMP

↓

🗄️ **PostgreSQL 16 · 1 conteneur · 4 bases isolées · Migrations Flyway** `auth_db` `catalog_db` `rental_db` `notification_db` `chat_db`

02 · ARCHITECTURE

Les services se parlent entre eux

🏠 → 📋 Annonce verrouillée ?

Avant de modifier ou supprimer une annonce, **catalog** demande à **rental** si elle a une candidature en cours. Si oui → refus.

🏠 → 🔑 Qui est le propriétaire ?

Quand on consulte une annonce, **catalog** demande à **auth** le nom du propriétaire pour l'afficher dans la fiche.

📋 → 🔔 Envoyer une notification

Quand une candidature est acceptée ou refusée, **rental** demande à **notification** d'envoyer une notification au locataire.

Si un service ne répond pas ?

Deux comportements selon l'enjeu :

Bloquant — vérification du lock : on refuse par précaution si rental est indisponible.

Optionnel — nom du propriétaire, notification : l'action réussit quand même, sans les données secondaires.

Principe de base

Chaque service reste **indépendant** — il n'accède jamais directement à la base de données d'un autre. Tout passe par des appels entre services.

03 · SÉCURITÉ

Un seul point de sécurité

1. L'utilisateur se connecte → **auth-service** génère un token signé
2. Le token est inclus dans chaque requête vers l'**API Gateway**
3. La gateway vérifie le token, puis transmet les infos utilisateur aux services
4. Les services reçoivent `X-User-Id` et `X-User-Email` — sans re-vérifier

Token de courte durée · 15 min

L'accès expire vite. Un **token de renouvellement** (7 jours) permet de reconnecter automatiquement l'utilisateur, sans qu'il s'en aperçoive.

Les services internes sont invisibles

Seule la gateway est accessible depuis l'extérieur. Toutes les routes `/internal/**` sont bloquées — elles n'existent que pour les appels entre services.

Renouvellement transparent

Le frontend renouvelle le token automatiquement en arrière-plan. L'utilisateur reste connecté sans interruption.

04 · FONCTIONNALITÉS

Un compte, deux modes

Mode Locataire

Basculer via le toggle dans le header

- ✓ Rechercher des annonces par ville, type, prix
- ✓ Consulter le détail avec les infos du propriétaire
- ✓ Soumettre une **candidature** (revenus, message)
- ✓ Suivre ses candidatures : EN ATTENTE → ACCEPTÉE / REFUSÉE
- ✓ Recevoir une **notification** à chaque décision

Mode Propriétaire

Même compte, même session

- ✓ Créer, modifier, supprimer une annonce
- ✓ Badge 🔒 **Verrouillée** si une candidature est en cours
- ✓ Consulter les candidatures reçues
- ✓ **Accepter ou refuser** une candidature
- ✓ Notification à chaque nouvelle candidature

Comptes de démo

`alice@loxia.dev` (locataire) · `bob@loxia.dev` (propriétaire) · mot de passe : `password123`

04 · RÈGLE MÉTIER CLÉ

L'annonce verrouillée

📌 **Une annonce avec une candidature** EN ATTENTE **ou** ACCEPTÉE **ne peut plus être modifiée ni supprimée.**

🏠

catalog-service

`PUT /api/catalog/listings/{id}`

`DELETE /api/catalog/listings/{id}`

↓ vérifie le lock avant d'agir
↓ `409 Conflict` si verrouillé

→ Est-ce que cette annonce est verrouillée ?

⇄

← oui / non

📋

rental-service

```
SELECT COUNT(*) > 0
FROM applications
WHERE listing_id = ?
AND status IN
  ('PENDING', 'ACCEPTED')
```

Si le service ne répond pas

La modification est **bloquée par précaution**. Mieux vaut refuser que corrompre les données.

Vérification en lot

Sur les listes, **une seule requête** vérifie tous les locks d'un coup — pas de surcharge réseau.

05 · STACK TECHNIQUE

Technologies choisies

Backend · Java 21

5 microservices + 1 gateway

FRAMEWORK

- Spring Boot 3.3
- Spring Web MVC
- Spring Security 6

DONNÉES

- Spring Data JPA
- Flyway
- PostgreSQL 16

INFRASTRUCTURE

- Spring Cloud Gateway
- Docker
- WebSocket STOMP
- Lombok
- JWT (HS256)

Frontend · React 18

SPA TypeScript · Nginx

UI & STYLE

- React 18
- TypeScript
- Tailwind CSS 4
- shadcn/ui

DONNÉES & ÉTAT

- TanStack Query v5
- Zustand
- Axios

FORMULAIRES & NAVIGATION

- React Hook Form
- Zod
- React Router v6
- react-leaflet
- SockJS · STOMP.js

06 · INFRASTRUCTURE

Docker Compose — tout en un

ACCESSIBLES DEPUIS LE NAVIGATEUR

🌐 **Frontend**
`localhost:3000`

🔒 **API Gateway**
`localhost:8080`

🐳 RÉSEAU INTERNE — INACCESSIBLE DEPUIS L'EXTÉRIEUR

🗄️ loxia-db	🔑 auth	🏠 catalog	📋 rental	🔔 notif	💬 chat
PG 16	:8081	:8082	:8083	:8084	:8085
	✓	✓	✓	✓	✓

Démarrage ordonné

Chaque service attend que la base de données soit **prête** avant de démarrer. La structure des tables est créée automatiquement.

Configuration par environnement

Un fichier `application.yml` pour le dev local, un autre pour Docker. Les secrets (`JWT_SECRET`, mots de passe) sont dans un fichier `.env` non versionné.

Lancer l'application

```
docker compose up -d --build
```

07 · DÉMONSTRATION

5 minutes de démo

PARCOURS

1. Naviguer **sans compte** — rechercher des annonces par filtres
2. Se connecter en tant qu'**alice** → candidater à une annonce
3. Se connecter en tant que **bob** → voir le badge 🔒 sur l'annonce
4. Tenter de **modifier l'annonce** → refus automatique
5. **Accepter la candidature** → alice reçoit une notification 🔔

🌐 **Frontend**
`localhost:3000`

🔒 **API**
`localhost:8080`

Comptes de démo

Alice — `alice@loxia.dev` · locataire
Bob — `bob@loxia.dev` · 3 annonces
Mot de passe : `password123`

Lancer la démo

```
docker compose up -d --build
./scripts/seed.sh
```

CONCLUSION

Ce que Loxia démontre

🏗️ Architecture distribuée réelle

5 services indépendants, chacun avec sa propre base — isolation totale des données.

⚛️ Frontend moderne

React 18, gestion du cache serveur, reconnexion automatique, formulaires validés.

🔒 Sécurité centralisée

Un seul point de validation JWT. Les services internes ne sont jamais exposés.

🐳 Déployable en une commande

Docker Compose orchestre les 8 conteneurs avec démarrage ordonné et données de démo.

📐 Bonnes pratiques Java

DTOs séparés, injection par constructeur, migrations Flyway, tests unitaires.

📚 Entièrement documenté

Architecture, décisions techniques, maquettes, script de démo — prêt à onboarder.

v1.0	6	16	1 cmd
Version stable	Microservices	Écrans maquettés	Pour tout lancer

Loxia

Location immobilière · Architecture Microservices · Java 21 + React 18

Ugo Durand · Jonas Obrun · Nicolas Ramirez · Cyril Grosjean · Ibrahim Khan · Tarek El Missiry

Spring Boot 3.3 | Spring Cloud Gateway | PostgreSQL 16 | React 18

TanStack Query | Zustand | shadcn/ui | Docker Compose | JWT HS256 | Flyway

localhost:3000 · localhost:8080 · `docker compose up -d --build`